File Handling in Java

Introduction to Java File Operations

Presenter Names:-

Priyanshi

Sukriti

Rama

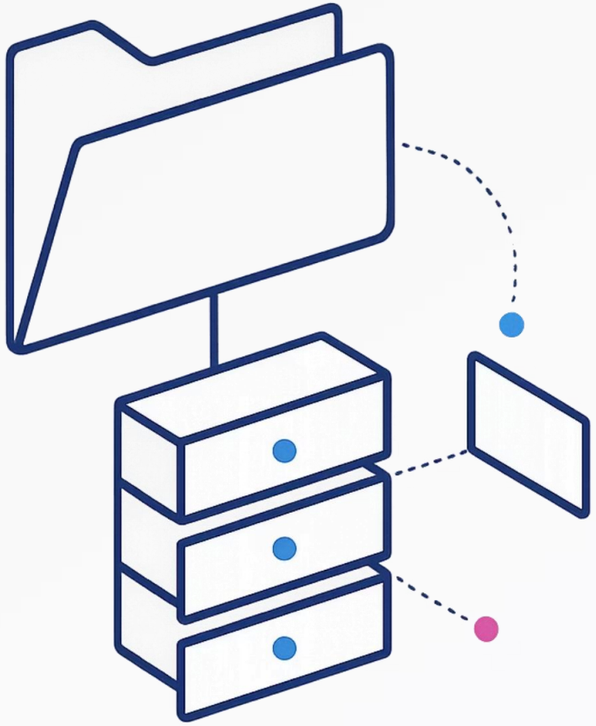
Shalu

Suchita

What is File Handling?

File handling is the process of reading, writing, creating, and manipulating files through programming code. It enables programs to store data permanently rather than keeping it in temporary memory.

In Java, file handling allows your programs to interact with the file system, saving data that persists even after the program terminates. This is fundamental for building applications that need to store user preferences, log activities, or maintain databases.



Why File Handling Matters

Data Persistence

Files store data permanently, unlike variables that lose information when programs end.

User Preferences

Save application settings and user configurations between sessions.

Logging

Record program events, errors, and debugging information for analysis.

Data Exchange

Share information between different programs through structured files.

Java File Handling Basics

Essential Components

Java provides a set of built-in classes for working with files and text data. These classes help you create, read, write, and manage files efficiently.

- **File** — Represents a file or directory path and lets you check, create, delete, or inspect files.
- **FileReader** — Reads character data from a file.
- **FileWriter** — Writes character data to a file.
- **BufferedReader** — Reads text efficiently by buffering input, making it ideal for reading lines.
- **BufferedWriter** — Writes text efficiently by buffering output before saving it to a file.

Together, these classes form the foundation of basic file handling in Java, especially when working with text-based files.

Creating Files in Java

Creating a file means making a new file on your computer so you can save data in it.

The main Java class for this is the **File** class.

Here is a simple example:

```
import java.io.File;
import java.io.IOException;

public class Main {
  public static void main(String[] args) {
    try {
      File file = new File("notes.txt");

      if (file.createNewFile()) {
        System.out.println("File created: " + file.getName());
      } else {
        System.out.println("File already exists.");
      }
    } catch (IOException e) {
      System.out.println("An error occurred.");
      e.printStackTrace();
    }
  }
}
```

Reading Data from Files

FileReader

Reads character data from files using character streams.

BufferedReader

Wraps FileReader to read text efficiently with buffering.

readLine()

Reads entire lines of text until end of file is reached.

Implementation Example

```
import java.io.*;

public class ReadFile {
 public static void main(String[] args) {
 try (BufferedReader br = new BufferedReader(
 new FileReader("example.txt"))) {
 String line;
 while ((line = br.readLine()) != null) {
 System.out.println(line);
 }
 } catch (IOException e) {
 System.out.println("Error: " + e.getMessage());
 }
 }
}
```

Writing Data to Files

FileWriter & BufferedWriter

FileWriter writes character data directly to a file for simple text output. **BufferedWriter** wraps a FileWriter and improves performance by buffering data in memory before writing it out.

Practical example

```
import java.io.*;

public class WriteFile {
  public static void main(String[] args) {
    try (BufferedWriter bw = new BufferedWriter(
        new FileWriter("output.txt"))) {
      bw.write("Hello, file!");
      bw.newLine();
      bw.write("This is the second line.");
    } catch (IOException e) {
      System.out.println("Error: " + e.getMessage());
    }
  }
}
```

Appending Data to Files

Append Mode

Open file in append mode by passing `true` as second parameter to `FileWriter` constructor. Data is added to end of existing content.

Advantages

- Preserve existing data
- Log multiple entries
- Build files incrementally
- Combine information efficiently

Usage Example

```
BufferedWriter bw = new BufferedWriter(
 new FileWriter("log.txt", true));
bw.write("New log entry");
bw.newLine();
bw.close();
```

Exception Handling in File Operations

Why Exception Handling?

File operations can fail due to various reasons: file not found, permission denied, disk full, or I/O errors.

Conclusion & Key Takeaways

01

File Handling Foundation

Master reading, writing, creating, and manipulating files using Java's `java.io` package.

02

Essential Classes

Learn `File`, `FileReader`, `FileWriter`, `BufferedReader`, and `BufferedWriter` for comprehensive file operations.

03

Exception Management

Always implement proper exception handling with try-catch blocks to handle `IOException` gracefully.

04

Real-World Applications

Apply file handling in logging systems, configuration files, data persistence, and file-based databases.

File handling is fundamental for building robust Java applications that require data persistence and file system interaction.